

Java漏洞原理与实战

0x01、基本概念

1、序列化与反序列化

(1)序列化:

将对象写入IO流中，ObjectOutputStream类的writeObject()方法可以实现序列化

(2)反序列化:

从IO流中恢复对象，ObjectInputStream类的readObject()方法用于反序列化

(3)意义:

序列化机制允许将实现序列化的Java对象转换为字节序列，这些字节序列可以保存到磁盘上，或通过网络传输，以达到以后恢复成原来的对象。序列化机制使得对象可以脱离程序的运行而独立存在



(4)序列化与反序列化

序列化与反序列化是让Java对象脱离Java运行环境的一种手段，可以有效的实现多平台之间的通信、对象持久化储存。主要应用在以下场景:

HTTP:多平台之间的通信，管理等，也可以用于流量带外

RMI:是Java的一组拥护开发分布式应用程序的API，实现了不同操作系统之间程序的方法调用。值得注意的是，RMI的传输100%基于反序列化，Java RMI的默认端口是1099端口

JMX:JMX是一套标准的代理和服务，用户可以在任何Java应用程序中使用这些代理和服务实现管理，中间件weblogic的管理页面就是基于JMX开发的，而JBoss则整个系统都基于JMX框架

(5)Java代码审计思路

如果是Java原生类，则需要入口类readObject方法，同时实现了序列化接口，使其可以进行有效的反序列化，此时如果存在DNS解析，或者实现反序列化（利用Runtime对象进行类反射操作）

需要有最终的执行函数(可以执行代码或者命令)，比如Runtime.getRuntime().exec，ProcessBuilder().start，getHostAddress，文件读写...等等，这些函数需要自己平常去收集，这样审计起来会更得心应手

2、Java类反射机制

(1) 反射机制的作用:通过Java语言中的反射机制可以操作字节码文件(可以读和修改字节码文件)，可以通过另外的方式调用到类的属性和方法，甚至私有属性和方法

(2) 反射机制的相关类在java.lang.reflect.*包下面

(3) 反射机制的相关类有哪些:Constructor、Field、Method、Class等类

(4) java.lang.Class代表字节码文件，代表整个类

(5) java.lang.reflect.Method代表字节码中的方法字节码，代表类中的方法java.lang.reflect.Constructor代表字节码中的构造方法字节码，代表类中的构造方法java.lang.reflect.Field代表字节码中的属性字节码，代表类中的属性

```
public class RunCalc {
    public static void main(String[] args) throws Exception{
        // 正常运行一个应用程序
        Runtime calc = Runtime.getRuntime();
        calc.exec("calc");

        // 使用类反射机制运行一个应用 程序
        Class clazz = Class.forName("java.lang.Runtime");    // 也可以使用 java.lang.Runtime.class获取
        Object obj = clazz.getMethod("getRuntime", null).invoke(clazz,null);
        clazz.getMethod("exec",String.class).invoke(obj, "calc.exe");
    }
}
```

(6) Java中为什么要使用反射机制，直接创建对象不是更方便？

如果有多个类，每个用户所需求的对象不同，直接创建对象，就要不断的去new一个对象，非常不灵活。而Java反射机制，在运行时确定类型，绑定对象，动态编译最大限度发挥了Java的灵活性

(7) 获取成员变量

```
Field[] field = c.getDeclaredFields()    // getFields则获取public属性
```

(8) 获取并调用方法

```
Method[] method = c.getDeclaredMethods()    // getMethods则获取public方法
```

(9) 获取构造方法

```
Constructor[] constructor = c.getDeclaredConstructors();    // getConstructors则获取所有构造方法
```

(10) 访问私有属性

```
f.setAccessible(true);
```

0x02、类反射机制实践

```
package com.woniu.vul;

import java.lang.reflect.Constructor;
```

```

import java.lang.reflect.Field;
import java.lang.reflect.Method;

class Test{
    public String name = "蜗牛学苑";
    public int age = 8;
    private String addr = "西安";
    private int price = 10000;

    public Test() {

    }

    public Test(int price){
        this.price = price;
    }

    public int setPrice(int price){
        System.out.println("新价格为:" + price);
        return price;
    }

    public int getPrice(){
        return this.price;
    }

    private void getAddr(){
        System.out.println("私有方法:" + addr);
    }

}

public class Reflect {
    public static void main(String[] args) throws Exception {
        /*Test t = new Test();
        System.out.println(t.getPrice());
        Test t1 = new Test(15000);
        System.out.println(t1.getPrice());
        System.out.println(t.name);
        */

        //使用反射机制实现属性和方法的调用（包括构造方法）
        //使用Class.forName可以获取到类本身，在JVM中动态加载Test类
        //Class clazz = Class.forName("com.woniu.vul.Test");
        //Class clazz = Test.class;
        //使用new Instance进行实例化
        //Test t = (Test) clazz.newInstance();
        //System.out.println(t.getPrice());

        //Object o = clazz.newInstance(); //实例化动态加载的类，类型必须是Object

        //
        Method m1 = clazz.getMethod("getPrice");

```

```
//      int price1 = (int)m1.invoke(o,null);
//      System.out.println(price1);
//
//      Method m2 = clazz.getMethod("setPrice",int.class);
//      int price2 = (int)m2.invoke(o,15000);
//      System.out.println(price2);

//调用price私有属性和getAddr私有方法，getFiled只能调用公有属性，getDeclareField才能
调私有属性
//Field f1 = clazz.getDeclaredField("price");
//f1.setAccessible(true); //设置私有属性可访问
//System.out.println(f1.get(o));

//getMethod只能调用公有方法，而getDeclareMethod才能调用私有方法
//Method m1 = clazz.getDeclaredMethod("getAddr");
//m1.setAccessible(true);
//m1.invoke(o,null);

//构造方法如果有参数，怎么办？
Class clazz = Class.forName("com.woniu.vul.Test");
Constructor c = clazz.getConstructor(int.class); //获取到一个带参数的构造器
Object o = c.newInstance(10); //用构造器去构造一个动态加载的类

Method m1 = clazz.getDeclaredMethod("getPrice");
int price = (int) m1.invoke(o,null);
System.out.println(price);

//遍历所有方法或操作
Method[] methods = clazz.getDeclaredMethods();
for (Method method : methods) {
    System.out.println(method + "    " + method.getName() + "    " +
method.getModifiers());
}

Field[] fields = clazz.getDeclaredFields();
for (Field field : fields) {
    System.out.println(field + "    " + field.getName() + "    " +
field.getModifiers());
}
}
}
```

0x03、序列化与反序列化

1、序列化的方式

如果需要将某个对象保存到磁盘上或者通过网络传输，那么这个类应该实现 `Serializable` 接口或者 `Externalizable` 接口之一。

使用到JDK中关键类：`ObjectOutputStream` (对象输出流) 和 `ObjectInputStream` (对象输入流)`ObjectOutputStream` 类中:通过使用 `writeObject(Object object)` 方法，将对象以二进制格式进行写入。

`ObjectInputStream` 类中:通过使用 `readObject()` 方法，从输入流中读取二进制流，转换成对象。

Transient关键字序列化的时候不会序列化**Transient**关键字修饰的变量（`private transient String phone;`），这个关键字不能修饰类和方法**Static**。静态变量也不会被序列化。

serialVersionUID：这里是指序列化的版本号，版本不一致会导致抛出错误，并且拒绝载入序列化与反序列化。

序列化必须满足以下几个条件：

- 1、实现 `Serializable` 接口或 `Externalizable` 接口
- 2、当前类提供一个全局常量 `serialVersionUID`
- 3、必须保证其内部所有属性也必须是可序列化的（默认情况下，基本数据类型可序列化）
- 4、`ObjectInputStream` 和 `ObjectOutputStream` 不能序列化 `static` 和 `transient` 修饰的成员变量

序列化的实现代码

```
package com.woniu.vul;

import java.io.*;

class Student implements Serializable {
    public String name = "";
    public int id = 0;
    public String phone = "";

    public Student(){
        System.out.println("构造方法运行");
    }

    public void study(){
        System.out.println("学生正在学习");
    }
    public void sleep(){
        System.out.println("学生正在休息");
    }
}

public class Unserial {
    public void serial() throws Exception {
        Student s = new Student();
        s.name = "张三";
        s.id = 12345;
        s.phone = "188123456786";
        FileOutputStream fos = new FileOutputStream("./data/student.ser");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(s);
    }

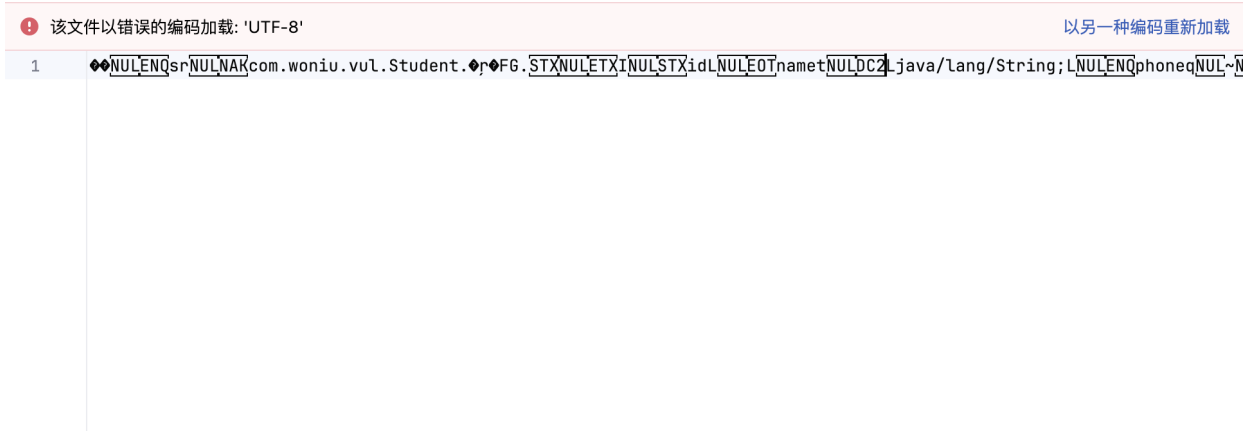
    public void unserial() throws Exception {
        FileInputStream fis = new FileInputStream("./data/student.ser");
        ObjectInputStream ois = new ObjectInputStream(fis);
    }
}
```

```

        Student obj =(Student) ois.readObject();
        System.out.println(obj.name);
        obj.study();
    }
    public static void main(String[] args) throws Exception {
        Unserial us = new Unserial();
        //us.serial();
        us.unserial();
    }
}

```

序列化的内容如下:



然后进行反序列化, 切记不要去改我们的序列化内容, 不然无法反序列化回去

```

/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents/Home/bin/java -javaagent:/Application
张三
学生正在学习

进程已结束, 退出代码为 0

```

我们再补充两个小问题

正常情况下, 高亮部分是可以被序列化的, 但是如果在这之前加上transient来修饰的话就无法序列化

```
class Student implements Serializable { 4个用法
```

```
    public String name = "";
```

```
    public int id = 0; 1个用法
```

```
    public String phone = ""; 1个用法
```

```
    transient public int price = 0; 0个用法
```

```
    public Student(){ 1个用法
```

```
        System.out.println("构造方法运行");
```

```
    }
```

高亮部分的意思就是定义一个序列化版本的编号，也就是唯一标识

```
import java.io.*;
```

```
class Student implements Serializable { 4个用法
```

```
    private static final long serialVersionUID = 1122334455L; 0个用法
```

```
    public String name = "";
```

```
    public String phone = ""; 1个用法
```

```
    transient public int price = 0; //transient修饰的变量不能被序列化 0个用法
```

```
    public Student(){ 1个用法
```

```
        System.out.println("构造方法运行");
```

```
    }
```

这个标识是用来干嘛的

接下来我们看看

我们重新反序列化看看效果

报错信息告诉我们是一个不可用的类

```
Unserial x
/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents/Home/bin/java
Exception in thread "main" java.io.InvalidClassException: Create breakpoint : com.woniu.vul
    at java.base/java.io.ObjectStreamClass.initNonProxy(ObjectStreamClass.java:598)
    at java.base/java.io.ObjectInputStream.readNonProxyDesc(ObjectInputStream.java:206)
    at java.base/java.io.ObjectInputStream.readClassDesc(ObjectInputStream.java:1912)
```

为什么不可用，让我们继续看报错信息

序列化的时候类的ID是前面的那一个，但是反序列化的时候类的ID是后面那一个，序列化和反序列化的时候标识号是不一样的，意思就是这个类并不是我们需要反序列化的类

```
IDEA.app/Contents/lib/idea_rt.jar=56940 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
: stream classdesc serialVersionUID = 3377072551707952942, local class serialVersionUID = 1122334455
```

我们将代码中的ID改为其序列化时候原本的ID，那我们就可以完成反序列化的操作了

```
import java.io.*;

class Student implements Serializable { 4 个用法
    private static final long serialVersionUID = 3377072551707952942L; 0 个用法
    public String name = "";
    public int id = 0; 1 个用法
    public String phone = ""; 1 个用法
    //transient public int price = 0; //transient修饰的变量不能被序列化
}
```

```
/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents,
张三
学生正在学习

进程已结束，退出代码为 0
```

也就是说这个类在序列化的时候会记录下其类的标识UID

接下来我们看看反序列化产生的机制

首先这个序列化对象一旦有重写的方法，那我们在反序列化的时候会优先调用重写的readObject

```
}
//重写readObject方法，在反序列化时会优先调用，如果该方法存在可利用点，则可以自动触发
private void readObject(ObjectInputStream ois) throws Exception{ 0 个用法
    ois.defaultReadObject();
    System.out.println("正在被反序列化.");
}
}
```



```

public void unserial() throws Exception { 1个用法
    FileInputStream fis = new FileInputStream( name: "./data/student.ser");
    ObjectInputStream ois = new ObjectInputStream(fis);
    Student obj =(Student) ois.readObject();
    System.out.println(obj.name);
    obj.study();
}

```

因为我们重写了readObject，所以就会先调用readObject，这就是Java反序列化的起点，也是唯一的起点

也就是说Java反序列化漏洞能够被利用，我们得有一个最基本的前提，就是目标类必须重写readObject方法，只有这样，代码才会被自动调用，否则就没有起点

如果不重写readObject方法的话，就不会发生Java反序列化漏洞

而我们的代码已经重写了readObject方法，所以我们可以对其利用

我们可以直接在重写方法的下面加上终点，也就是攻击者想要达到的效果，有始有终，整个攻击链才算完整

```

//重写readObject方法，在反序列化时会优先调用，如果该方法存在可利用点，则可以自动触发
private void readObject(ObjectInputStream ois) throws Exception{ 0个用法
    ois.defaultReadObject();
    System.out.println("正在被反序列化.");
    Runtime.getRuntime().exec( command: "ifconfig");
}
}

```

当然我们也可以使用类反射机制的手段去执行命令

因为getRuntime的实例不是纯粹的new出来的，而是通过调用getRuntime这个方法来获取其实例的，然后再通过这个实例去调用exec

加载java.lang.Runtime类

获取Runtime类中的getRuntime方法

调用Runtime方法，获取Runtime类的实例

获取Runtime类中的exec(string)方法

调用exec(String)方法，运行外部命令ifconfig

```

Class clazz = Class.forName( className: "java.lang.Runtime");
Method m1 = clazz.getDeclaredMethod( name: "getRuntime");
Object obj = m1.invoke(clazz, ...args: null);
Method m2 = clazz.getDeclaredMethod( name: "exec",String.class);
m2.invoke(obj, ...args: "ifconfig");
}
}

```

运行代码，发现没有报错，说明应该是利用成功了，我不知道为啥不会显示执行ifconfig命令的内容，如果是Windows的话，可以将ifconfig改为calc.exe，大概率会显示出计算器

```
/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents/Home/bin/java -javaagent:/Applications/IntelliJ
正在被反序列化。
张三
学生正在学习

进程已结束，退出代码为 0
```

当然为了执行命令，不仅仅只有Runtime，还有ProcessBuilder

接下来我们看看其反射的调用

根据正常的调用来构造反射

先使用Class.forName这个方法加载java.lang.ProcessBuilder这个类

然后使用Class对象的getConstructor来获取Processbuilder类的构造函数

接着使用Constructor对象的新Instance方法来创建ProcessBuilder的实例

然后使用Class对象的getMethod方法去获取ProcessBuilder中的start方法

最后使用Method对象的invoke方法去调用start方法

```
//      ProcessBuilder pb = new ProcessBuilder("calc.exe");
//      pb.start();

Class clazz = Class.forName( className: "java.lang.ProcessBuilder");
Constructor c = clazz.getConstructor(String.class);
Object obj = c.newInstance( ...initargs: "calc.exe");
Method m = clazz.getMethod( name: "start");
m.invoke(obj, ...args: null);
}
}
```

然后运行，发现报错，是类型出现了错误

```
/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents/Home/bin/java -javaagent:/Applications/IntelliJ
正在被反序列化。
Exception in thread "main" java.io.IOException: Create breakpoint : unexpected exception type
    at java.base/java.io.ObjectStreamClass.throwMiscException(ObjectStreamClass.java:1592)
    at java.base/java.io.ObjectStreamClass.invokeReadObject(ObjectStreamClass.java:1116)
    at java.base/java.io.ObjectInputStream.readSerialData(ObjectInputStream.java:2429)
    at java.base/java.io.ObjectInputStream.readOrdinaryObject(ObjectInputStream.java:2269)
    at java.base/java.io.ObjectInputStream.readObject0(ObjectInputStream.java:1747)
```

我们先去看看ProcessBuilder的构造方法，它不是严格意义上的String，是String...（可变长的字符串）如果是whoami /user这条命令的话，我们得写到两个字符串里面，在Java中，对于可变长的字符串是将其放入到数组当中去

© java.lang.ProcessBuilder

```
public ProcessBuilder(  
    @NotNull String... command  
)
```

然后我们将其修改为String[].class

```
Class clazz = Class.forName( className: "java.lang.ProcessBuilder");  
Constructor c = clazz.getConstructor(String[].class);  
Object obj = c.newInstance( ...initargs: "calc.exe");  
Method m = clazz.getMethod( name: "start");  
m.invoke(obj, ...args: null);
```

然后继续运行，然后还是报错说类型不匹配

/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents/Home/bin/java -javaagent:/Applications/I
正在被反序列化。

```
Exception in thread "main" java.lang.IllegalArgumentException Create breakpoint : argument type mismatch <1 个内部行>  
    at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:502) <1 个内部行>  
    at com.woniu.vul.Student.readObject(Unserial.java:43) <2 个内部行>  
    at java.base/java.io.ObjectStreamClass.invokeReadObject(ObjectStreamClass.java:1108)  
    at java.base/java.io.ObjectInputStream.readSerialData(ObjectInputStream.java:2429)  
    at java.base/java.io.ObjectInputStream.readOrdinaryObject(ObjectInputStream.java:2269)
```

就是因为我们上面定义的是数组，下面是字符串，所以会报错

```
Constructor c = clazz.getConstructor(String[].class);  
Object obj = c.newInstance( ...initargs: "calc.exe");
```

所以我们要将下面的类型转换为数组类型就可以了，也就是将其放到数组中就可以了，如下

```
String[] cmd = {"calc.exe"};  
Object obj = c.newInstance(cmd);  
Method m = clazz.getMethod( name: "start");
```

继续运行，发现还是报错，报错信息还是类型不匹配

```
/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents/Home/bin/java -javaagent:/Applications/IntelliJ I  
正在被反序列化。  
> Exception in thread "main" java.lang.IllegalArgumentException Create breakpoint : argument type mismatch k1 个内部行  
>    at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:502) <1 个内部行>  
>    at com.woniu.vul.Student.readObject(Unserial.java:44) <2 个内部行>  
    at java.base/java.io.ObjectStreamClass.invokeReadObject(ObjectStreamClass.java:1108)  
    at java.base/java.io.ObjectInputStream.readSerialData(ObjectInputStream.java:2429)  
    at java.base/java.io.ObjectInputStream.readOrdinaryObject(ObjectInputStream.java:2269)
```

我们去看看newInstance的构造方法，发现还是一个数组，它的类型是数组，数组里面的参数还是一个数组

```
© java.lang.reflect.Constructor<T>
```

```
@NotNull ↗
```

```
public T newInstance(  
    Object... initargs  
)  
throws InstantiationException, IllegalAccessException, IllegalArgumentException, java.lang.reflect.  
InvocationTargetException
```

所以这个cmd的类型要定义成二维数组，二维数组只需加两个{}即可，如下

```
Constructor c = clazz.getConstructor(String[].class);  
String[][] cmd = {"calc.exe"};  
Object obj = c.newInstance(cmd);  
Method m = clazz.getMethod("start");  
m.invoke(obj, null);
```

然后去运行，发现没有报错，但是也没有回显命令的内容，应该是电脑的问题，如果是Windows的话在命令那一块改为calc.exe就可以打开计算器了

```
/Users/zhangjie/Library/Java/JavaVirtualMachines/openjdk-22.0.1/Contents/Home/bin/java -javaagent:/App  
正在被反序列化。  
张三  
学生正在学习  
  
进程已结束，退出代码为 0
```